TP 3

Les variables d'environnement et les programmes Set-UID

1. Introduction

L'objectif de ce TP est de permettre aux étudiants de comprendre comment les variables d'environnement affectent le comportement des programmes et du système. Les variables d'environnement sont un ensemble de valeurs dynamiques nommées qui peuvent affecter la façon dont les processus en cours d'exécution se comportent sur un ordinateur. Elles sont utilisées par la plupart des systèmes d'exploitation, depuis qu'elles ont été introduites dans Unix en 1979. Bien que les variables d'environnement affectent le comportement des programmes, la manière dont elles y parviennent n'est pas bien comprise par de nombreux programmeurs. Par conséquent, si un programme utilise des variables d'environnement, mais que le programmeur ne sait pas qu'elles sont utilisées, le programme peut présenter des vulnérabilités.

Dans ce TP, nous essayons de comprendre comment ces variables fonctionnent, comment elles sont propagées du processus parent au processus fils, et comment elles affectent les comportements du système/programme ; notamment les programmes Set-UID, qui sont habituellement des programmes privilégiés.

Environnement de laboratoire : VM SEED Ubuntu 20.04.

2. Application

Les fichiers nécessaires pour ce laboratoire sont inclus dans le dossier `Labsetup.zip`.

2.1. Manipulation des variables d'environnement

Dans cette activité, nous étudions les commandes qui peuvent être utilisées pour définir et supprimer les variables d'environnement. Nous utilisons Bash dans le compte seed. Le shell par défaut qu'un utilisateur utilise est défini dans le fichier /etc/passwd (le dernier champ de chaque entrée).

- Commande pour voir le shell par défaut : cat /etc/passwd | grep seed
- Utilisez la commande *printenv* ou *env* pour imprimer les variables d'environnement. Si vous êtes intéressé par certaines variables d'environnement particulières, telles que PWD, vous pouvez utiliser la commande "*printenv PWD*" ou "*env | grep PWD*".
- Utilisez la commande *export* pour définir et exporter des variables d'environnement.
- Utilisez la commande *unset* pour supprimer des variables d'environnement.

Exemple :

```
export MYVAR="Test"    # MYVAR est une var exportée
bash                   # Ouvre un nouveau shell (un sous-shell).
echo $MYVAR            # Affiche "Test"
```

2.2. Passage des variables d'environnement du processus parent au processus fils

Dans cette activité, nous étudions comment un processus fils obtient ses variables d'environnement de son parent. Sous Unix, la fonction **fork()** crée un nouveau processus en dupliquant le processus appelant. Le nouveau processus, appelé processus fils, est une copie exacte du processus appelant, appelé parent. Cependant, plusieurs choses ne sont pas héritées par le fils (veuillez consulter le manuel de fork() avec la commande : man fork).

Dans ce qui suit, nous allons découvrir si les variables d'environnement du parent sont héritées par le processus fils ou non.

Étape 1. Compilez et exécutez le programme suivant, et décrire votre observation. Le programme est dans le dossier Labsetup.

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
extern char **environ;
void printenv()
{
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
      printenv();          (1)
      exit(0);
    default:  /* parent process */
      // printenv();        (2)
      exit(0);
  }
}
```

Code 1 : myprintenv.c

Étape 2. Maintenant, commentez l'instruction printenv() dans le cas du processus fils (Ligne 1), et décommentez l'instruction printenv() dans le cas du processus parent (Ligne 2).

Compilez et exécutez à nouveau le code. Sauvegardez la sortie dans un autre fichier.

Étape 3. Comparez la sortie de ces deux fichiers en utilisant la commande diff.

Que remarquez-vous ?

2.3. Variables d'environnement et execve()

Dans cette activité, nous verrons comment les variables d'environnement sont affectées lorsqu'un nouveau programme est exécuté via la fonction execve().

Cette fonction fait un appel système pour charger une nouvelle commande et l'exécuter, elle ne fait aucun retour sauf en cas d'erreur. Aucun nouveau processus n'est créé ; au lieu de cela, le texte, les données et la pile du processus appelant sont écrasés par ceux du programme chargé. Particulièrement, execve() exécute le nouveau programme à l'intérieur du processus appelant. Dans ce qui suit, nous nous intéressons à ce qui arrive aux variables d'environnement ; sont-elles automatiquement héritées par le nouveau programme ?

Étape 1. Compilez et exécutez le programme ci-dessous, et décrire votre observation.

Ce programme exécute simplement un programme appelé /usr/bin/env, qui affiche les variables d'environnement du processus actuel.

```
#include <unistd.h>
extern char **environ;
int main()
{ char *argv[2];
  argv[0] = "/usr/bin/env";
  argv[1] = NULL;
  execve("/usr/bin/env", argv, NULL);      (1)
  return 0 ;
}
```

Code 2 : myenv.c

Étape 2. Changez l'appel de execve() dans la ligne (1) par ce qui suit et décrivez votre observation.

```
execve("/usr/bin/env", argv, environ) ;
```

Étape 3. Expliquez comment le nouveau programme récupère son environnement.

2.4. Variables d'environnement et system()

La fonction system() est utilisée pour exécuter une commande, mais contrairement à execve(), qui exécute directement une commande, system() exécute en fait "/bin/sh -c command", c'est-à-dire qu'elle exécute /bin/sh, et demande au shell d'exécuter la commande. Par conséquent, en utilisant system(), les variables d'environnement du processus appelant sont transmises au nouveau programme /bin/sh.

Veuillez compiler et exécuter le programme suivant pour vérifier ceci.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
        system("/usr/bin/env");
        return 0 ;
}
```

Quelle est votre observation ?